National University of Computer and Emerging Sciences
Islamabad Campus

Parallel and Distributed Computing - (CS3006)

Final Exam

Course Instructors(s):
Dr. Muhammad Arshad Islam, Mr. Fahad Shafique, Mr. Muhammad Aadil Ur Rehman

Section(s): CS (A, B, C, D, E, F, G)

Total Time (Hrs):	**1**
Total Marks:	**55**
Total Questions:	**2**
Date:	June 03, 2026

Roll No	Course Section	Student Signature

Important Information: The use or possession of mobile phones and other electronic devices during the exam is strictly prohibited, whether the device is powered on or off. Do not bring mobile phones, smartwatches, tablets, or other electronic gadgets into the examination hall. If you are found with any prohibited devices, your exam may be cancelled, and further disciplinary action may be taken.
Do not write below this line.

Attempt all the questions

Part A: Objective

Instructions:

1. Read the question carefully, understand the question, and then attempt it.
2. All questions must be attempted on the same question paper in the space provided.
3. After asked to commence the exam, please verify that you have (**15**) different printed pages.
4. No sharing of calculators, notes, or any other material during the exam.
5. Use permanent ink pens only.

	I	II	**Total**
CLO #	1	1	
Total Marks	25	30	55
Marks Obtained			

[CLO 1: Demonstrate understanding of various concepts involved in parallel and distributed computer architectures.]

Question I . . . . . (25 Marks)

Answer the following short questions within the space provided.

(1) **(4 Marks)** A company is designing a new server. They have two options:

Option 1: A cc-NUMA machine with 128 cores, where memory access to the local node takes 80 ns and remote access takes 300 ns.

Option 2: A distributed memory cluster with 128 cores (each node has 4 cores and its own memory), where cores within a node share memory (UMA) but across nodes communication requires explicit message passing with 1 µs latency.

The server will run a database workload where each transaction reads and writes a small set of data, and the working set fits entirely within one node's memory. However, transactions are randomly assigned to any core. **Which option will likely provide better performance, and why? Justify with two distinct reasons based on architectural differences.**

Solution: $1 mark$ **Option 1** (cc-NUMA) will provide better performance.
$1.5 marks$ **Reason 1:** Because transactions are randomly assigned, remote data access is highly probable. Option 1 resolves remote fetches via a hardware interconnect in 300 ns, which is significantly faster than Option 2's 1 µs network latency.
$1.5 marks$ **Reason 2:** Option 1 uses transparent hardware cache coherence (load/store) for remote access. Option 2 requires explicit message passing, which adds massive software overhead (OS stack, packet assembly) on top of the network delay for small transaction reads.

(2) **(3 Marks)** A research lab is designing a new parallel computer for two different applications:

Application A: A weather simulation that updates a large 3D grid. Each grid point depends only on its immediate neighbours. The simulation runs for many time steps, and the entire grid is too large to fit in any single processor's cache. The workload is parallelised by dividing the grid into sub-grids, one per processor.

Application B: A database query engine that handles thousands of concurrent transactions. Each transaction reads and writes a small, random set of records. The entire database fits in the aggregate memory of the system. Transactions are independent, but they occasionally update shared counter variables.

The lab must choose between two architectures:

Feature	Architecture X	Architecture Y
Memory organisation	Uniform Memory Access (UMA) with a single shared bus	Cache-Coherent Non-Uniform Memory Access (cc-NUMA) with 4 nodes
Number of cores	32	32 (8 cores per node)
Local memory latency	100 ns (all accesses)	Local: 60 ns, Remote: 300 ns
Cache coherence	Hardware, broadcast on bus	Hardware, directory-based
Programming model	Shared memory (automatic coherence)	Shared memory (automatic coherence)

A student claims that "Architecture Y is always better because it has lower local latency and more scalable

interconnect." **Provide a counterexample using one of the two applications, explaining why Architecture X would be preferable.**

Solution: $1 mark$ **Application B** provides a counterexample.
$2 marks$ For this workload, Architecture X (UMA) is preferable because the frequent random updates to shared counters would cause excessive remote accesses and coherence traffic in cc-NUMA, negating the benefit of lower local latency. Thus, uniformity of access time can be more important than low local latency when data sharing is unpredictable.

(3) **(3 Marks)** When processing a large float array in a loop using AVX2 intrinsics, a programmer notices that unrolling the loop by 8 vectors (64 floats per iteration) does not improve performance compared to unrolling by 2 vectors. The CPU has 16 architectural vector registers (ymm0–ymm15). What is the most likely explanation?

Solution: The most likely explanation is **register spilling** due to high register pressure. With only 16 architectural vector registers (ymm0-ymm15), unrolling by 8 vectors demands too many concurrent registers to hold multiple operands, intermediate results, and loop constants. This forces the compiler to temporarily save (spill) register values to the slower stack memory, introducing memory overhead that completely negates the pipeline benefits of the deeper unroll.

(4) **(3 Marks)** A function signature uses restrict as in:

```
1    void axpy_simd_restrict(float * restrict y, const float * restrict x,
2                            float a, size_t N)
```

The measured performance sometimes improves compared to the same code without `restrict`. Why does `restrict` help even when the programmer already uses explicit SIMD intrinsics?

Solution: restrict tells the compiler that x and y do not overlap, allowing it to schedule loads and stores more aggressively and possibly combine memory operations. Without it, the compiler must assume that writing to y could change the values of x (if they overlap), forcing conservative ordering of loads and stores.

(5) **(3 Marks)** Why do DSLs like Halide or Tensor Comprehensions often produce more efficient code than general-purpose languages for their target domains?

Solution: They restrict expressiveness, allowing the compiler to make stronger assumptions about data dependences and memory access patterns, which enables architecture-specific optimisations (tiling, vectorisation, fusion).

Reasoning: DSLs are limited to a domain (e.g., image processing, dense linear algebra). This limitation allows the compiler to infer that there are no hidden dependences, that data is regularly structured, and that operations are side-effect-free. Consequently, aggressive architecture-specific transformations (loop fusion, tiling, vectorisation) become safe and effective.

(6) **(3 Marks)** During loop tiling optimization, what are the microarchitectural performance implications if the chosen tile size exceeds the L2 cache capacity but fits entirely within the L3 cache?

Solution: Exceeding the L2 cache capacity while fitting within the L3 cache causes **continuous L2 cache capacity misses and thrashing**, forcing the CPU to fetch data from the slower, shared L3 layer. While this configuration **avoids the catastrophic latency penalty of hitting main memory (DRAM)**, making it vastly superior to an untiled loop, it still introduces a 3x to 4x latency penalty per data access (roughly 40 clock cycles for L3 versus 12 for L2) that can stall the execution pipeline. Furthermore, because L3 is shared across cores, flooding the internal interconnect with L2 miss traffic can saturate ring or mesh networks, resulting in significantly lower throughput compared to a tightly optimized, L2-compliant tile size.

(7) **(6 Marks)** You are running an MPI simulation on 256 nodes. Each iteration updates a large grid. The simulation needs 10,000 iterations. System MTBF = 12 hours. Checkpoint overhead = 10 minutes (writing all ranks' data to a parallel file system). The simulation currently checkpoints every 100 iterations. Each iteration takes 30 seconds.

```
for (int iter = 1; iter <= ITERS; iter++) {
    compute_next_grid();
    if (iter % CHECKPOINT_INTERVAL == 0) {
        MPI_Barrier(MPI_COMM_WORLD);
        checkpoint_write();
    }
}
```

If a failure occurs exactly at iteration 250, and the last successful checkpoint was at iteration 200, how much work is lost (in seconds)? Also, Suggest a change to the checkpoint strategy that would reduce lost work without significantly increasing total runtime.

Solution: Work lost = iterations since last checkpoint = 250 − 200 = 50 iterations. Each iteration = 30 seconds → lost work = 50 × 30 = 1,500 seconds (25 minutes).

Increase checkpoint frequency (e.g., every 50 iterations). This reduces lost work (max 50 iterations instead of 100) but increases the number of checkpoints to 200, doubling overhead to 33.3 hours. The trade-off: lower loss at failure vs. higher overhead in failure-free runs.

or

Implement asynchronous (non-blocking) checkpointing using dedicated background I/O threads. Main compute nodes continue running the simulation without waiting for the 10-minute file system write to finish.

[CLO 1: Demonstrate understanding of various concepts involved in parallel and distributed computer architectures.]

Question II . . . . . (30 Marks)

Answer the following MCQs by filling the provided OMR Sheet provided at the end of this question paper as per instructions provided.

(1) **(1 Mark)** A processor with a single core can issue up to 4 instructions per cycle (superscalar width 4). A particular program has an instruction dependency graph with an average critical path length of 100 cycles and a total instruction count of 800. What is the best possible speedup over a scalar (1 instruction-per-cycle) processor for this program?

(A) No Speedup (B) 2 (C) 4
(D) 8 (E) 1.5

Solution: C

Reasoning: The scalar processor takes 800 cycles. With superscalar width 4, the ideal execution time is 800/4 = 200 cycles, giving speedup 4. The critical path does not limit because enough independent instructions exist.

(2) **(1 Mark)** A computer architect proposes to double the clock frequency of a single-core processor while keeping the same microarchitecture (same number of functional units, same pipeline depth). Which of the following consequences would most likely prevent this design from being commercially viable?

(A) Instruction-level parallelism would become the dominant bottleneck.
(B) The memory wall would cause the processor to stall for data more than half the time.
(C) The number of transistors would have to increase, violating Moore's Law.
(D) Power consumption would increase by a factor of roughly 4 to 8, exceeding thermal limits.
(E) None of the above seems correct in this case

> **Solution:** D
> Reasoning: Doubling frequency at constant voltage roughly doubles power, but often requires higher voltage, leading to exponential power increase (Power Wall), making it commercially unviable due to heat.

(3) **(1 Mark)** system has a single processor with a cache that can supply data at 100 GB/s. The main memory bandwidth is 20 GB/s. The processor runs a program that streams through a large array (much larger than cache) and performs a simple arithmetic operation per element. When the program is parallelised across 4 cores on the same chip, the speedup observed is only 1.6× instead of the ideal 4×. Which primary limitation from the overview best explains this?

(A) The power wall prevents all cores from running at full frequency simultaneously. (B) The memory wall as all cores share the same main memory bandwidth. (C) The thermal wall forces the cores to be throttled after a few seconds. (D) The ILP of the arithmetic operation is too low to benefit from multiple cores. (E) The operating system incurs high scheduling overhead when using more than 2 cores.

> **Solution:** B
> Reasoning: Memory wall – all cores share the same limited main memory bandwidth. A single core already saturates the bandwidth, so additional cores cannot feed data faster, severely limiting speedup.

(4) **(1 Mark)** In the early 2000s, processor designers shifted from building increasingly wide superscalar cores to replicating multiple simpler cores on the same die. Which of the following best explains why this shift occurred, given the fundamental limits discussed?

(A) Instruction-level parallelism (ILP) became harder to extract from typical workloads beyond a small window, while thread-level parallelism (TLP) from multiple software threads could be exploited by more cores.
(B) The number of transistors per chip stopped increasing, so designers could no longer afford wide superscalar logic.
(C) Operating systems could not schedule instructions efficiently for wide superscalar cores, but they could schedule threads across multiple cores.
(D) Memory bandwidth increased dramatically, making wide superscalar execution less beneficial relative to multiple cores.
(E) Programmers refused to write parallel code for wide superscalar cores, so companies switched to multicore to force parallelism.

> **Solution:** A
> Reasoning: Diminishing returns of ILP (beyond small superscalar width) combined with continued transistor density growth made replicating cores (exploiting TLP) more effective than building wider single cores.

(5) **(1 Mark)** Consider a cache-coherent Non-Uniform Memory Access (cc-NUMA) machine with two nodes. Processor P1 on Node A accesses a memory location that resides in Node B's local memory for the first time. Which sequence of memory hierarchy levels will the request visit, assuming L1 and L2 caches are private to each processor?

(A) P1's L1 → P1's L2 → main memory
(B) P1's L1 → P1's L2 → Node B's main memory → Node B's L2
(C) P1's L1 → P1's L2 → Node A's main memory → Node B's remote memory
(D) Node A's main memory → Node B's main memory → Node B's L2
(E) P1's L1 → P1's L2 → Node B's L2 → Node B's remote memory

Solution: C
In cc-NUMA: L1 cache → L2 cache (local to processor) → Main memory (local to node) → Remote memory.

(6) **(1 Mark)** Two systems are being compared:

System X: A shared memory SMP with 8 cores and a single bus.

System Y: A distributed memory cluster with 8 nodes, each with one core, connected by a fast network.

A scientist runs a simulation where each core must frequently access a large global data structure that is updated by all cores. Which system will likely suffer more from contention, and why?

(A) System X, because the single bus becomes a bottleneck when many cores try to access shared memory simultaneously.
(B) System Y, because the network latency is higher than the bus latency in System X.
(C) Both suffer equally because the total number of cores and the frequency of updates are the same.
(D) System Y, because each node has its own memory, requiring explicit message passing for every access.
(E) System X, because cache coherence traffic on the bus increases with frequent updates to shared data.

Solution: E
Classic SMP limitation, for distributed system obviously we would required some data communication effort like duplication.

(7) **(1 Mark)** A developer writes an AVX2 kernel that processes 32-bit floats using _mm256_load_ps and _mm256_store_ps. On a modern CPU (Intel Skylake or newer), the program runs correctly but occasionally shows small performance variations. The developer then switches to _mm256_loadu_ps and _mm256_storeu_ps without changing anything else. Which statement best describes the expected behaviour?

(A) The unaligned version will always be slower because it forces two cache line accesses for every load.
(B) The unaligned version may be equally fast when addresses happen to be 32-byte aligned, but can cause segmentation faults when misaligned.
(C) The aligned version may crash if the memory allocator returns a pointer that is not 32-byte aligned, the unaligned version avoids that risk with negligible performance difference on modern CPUs.
(D) The unaligned version will be faster because it removes the compiler's alignment checking overhead.
(E) Both versions will produce identical assembly code because the compiler automatically aligns all memory accesses.

Solution: C
Reasoning: _mm256_load_ps requires 32-byte alignment; passing a misaligned pointer causes a segmentation fault. _mm256_loadu_ps handles misalignment. On modern Intel CPUs (Haswell and later), unaligned loads that do not cross a cache line boundary incur no penalty.

(8) **(1 Mark)** A programmer optimises a memory-bound loop (streaming through large arrays) using AVX2 SIMD intrinsics. She first implements a simple SIMD version (variant 2). Then she unrolls the loop by processing 4 vectors per iteration (32 floats, variant 5). After measuring, she finds that the unrolled version shows almost no speedup. What is the most likely primary reason?

(A) The CPU's out-of-order execution already hides loop overhead; unrolling adds instruction cache pressure.
(B) The CPU's hardware prefetcher cannot handle the larger stride introduced by unrolling.
(C) The compiler already unrolled the loop automatically, so manual unrolling creates redundant code.
(D) Unrolling by 4 vectors causes register spilling, which increases memory traffic.
(E) The loop is already limited by main memory bandwidth; adding more arithmetic instructions per iteration does not make data arrive faster.

Solution: E
Reasoning: For a memory-bound kernel like AXPY (reads x and y, writes y, 12 bytes per element),

the bottleneck is DRAM bandwidth, not computation.

(9) **(1 Mark)** A developer has two functions that compute the dot product of two float vectors of length 256. Function A uses compiler auto-vectorisation with -O3 -mavx2. Function B uses hand-written AVX2 intrinsics with explicit loop unrolling. On an Intel Core i7-9700K, both produce correct results. Which statement about their performance is most likely true?

(A) Function B will always be faster because hand-tuned intrinsics outperform any compiler output.
(B) Function A will be faster because the compiler can optimise across function boundaries and use better scheduling.
(C) For a dot product of this size, the performance difference is likely negligible because the entire computation fits in L1/L2 cache.
(D) Function B will only be faster if the developer uses aligned loads; otherwise, the compiler's version wins.
(E) Function A cannot generate FMA instructions because auto-vectorisation does not support FMA.

Solution: C
Reasoning: Dot product of 256 floats involves 256 multiply-add operations. The data fits in L1 cache (256*4*2 = 2048 bytes plus a small accumulator). For such a small, compute-bound kernel, modern compilers generate nearly optimal SIMD code.

(10) **(1 Mark)** A programmer is deciding whether to use SIMD intrinsics for a function that processes a large array of 32-bit floats. The function performs a complex, non-linear operation that has many conditional branches inside the loop body. Which statement best predicts the outcome of using SIMD intrinsics for this function?

(A) SIMD intrinsics will improve performance significantly because they increase arithmetic throughput.
(B) SIMD intrinsics will likely not help because the conditional branches cause divergent execution, forcing the vector lanes to serialize or execute all branches.
(C) The performance will be identical to scalar because the compiler auto-vectorises branchy code efficiently.
(D) SIMD intrinsics will cause correctness issues because branching is not supported in vector registers.
(E) None of the above statements seems correct

Solution: B
Reasoning: SIMD (especially on CPUs) executes the same instruction across multiple data lanes. Conditional branches inside a loop lead to divergence: some lanes may need to take one path, others a different path.

(11) **(1 Mark)** A 4-thread OpenMP parallel region processes a loop of 1000 iterations using schedule(static, 1). How many distinct iterations does thread 0 execute?

(A) 250 (B) 1 (C) 1000
(D) 250 or 251 (depends on rounding) (E) All iterations that are multiples of 4

Solution: C
Not a work-sharing construct for

(12) **(1 Mark)** Consider the following OpenMP Program Snippet:

```
1 int a = 10;
2 #pragma omp parallel if(a<10) firstprivate(a)  num_threads(4)
3 {
4     a += omp_get_thread_num();
5 }
6 printf("%d\n", a);
```

What is printed?

(A) 10
(B) 10 + (0+1+2+3) = 16
(C) A random number between 10 and 16
(D) Compilation error
(E) Undefined, depends on which thread finishes last

Solution: A

(13) **(1 Mark)** A programmer wants to parallelise a nested loop:

```
1    for (int i = 0; i < M; i++)
2    for (int j = 0; j < N; j++)
3        C[i][j] = A[i][j] + B[i][j];
```

Which OpenMP directive gives the best load balance when M is small (e.g., 4) and N is very large (e.g., 1,000,000)?

(A) #pragma omp parallel for before the outer loop
(B) #pragma omp parallel for collapse(2)
(C) #pragma omp parallel for before the inner loop
(D) #pragma omp parallel for schedule(static,1) before the inner loop
(E) #pragma omp parallel with manual chunk distribution using if(i==0)

Solution: B

(14) **(1 Mark)** In a large-scale MPI application, a rank crashes due to a segmentation fault. With the default MPI error handler, what happens?

(A) The application pauses until the failed rank is restarted by the resource manager.
(B) Only the failed rank exits; all other ranks continue execution.
(C) The entire MPI job aborts immediately.
(D) MPI automatically recovers by re-spawning the failed rank.
(E) The communicator is marked as invalid, but other ranks can continue using a different communicator.

Solution: C Reasoning: Default MPI error handler is MPI_ERRORS_ARE_FATAL. Any error (including a rank crash) causes the entire job to abort. Option (E) is possible only if the user has changed the error handler to MPI_ERRORS_RETURN and implemented repair (e.g., ULFM).

(15) **(1 Mark)** Which of the following correctly describes the relationship between fault, error, and failure in the standard dependability taxonomy?

(A) A fault is a service failure; an error is the cause; a failure is the incorrect state.
(B) Fault and error are synonyms; failure is the service disruption.
(C) An error is a physical defect; a fault is an incorrect state; a failure is the detection mechanism.
(D) A fault is a physical defect; an error is an incorrect state; a failure is when service stops.
(E) A failure causes an error, which then causes a fault.

Solution: D

(16) **(1 Mark)** Halide separates the algorithm description from the schedule. Which of the following is a direct benefit of this separation?

(A) The algorithm can be written in a high-level, side-effect-free style, while the schedule specifies low-level optimisation decisions like tiling and vectorisation.
(B) The compiler can automatically generate the algorithm from the schedule, reducing programmer effort.
(C) It forces the programmer to write both parts in a single file, improving locality.

(D) It eliminates the need for parallelism because the schedule can be executed sequentially.
(E) The schedule determines the algorithm's correctness, so the algorithm can be ignored.

> **Solution:** A
> Reasoning: Halide's key innovation is decoupling "what to compute" (algorithm, declarative) from "how to compute" (schedule, including tiling, vectorisation, parallelisation). This allows optimisations without cluttering the algorithm code.

(17) **(1 Mark)** A programmer runs an OpenMP program on a 16-core, 64-byte cache line machine. The program updates adjacent elements of a float array partial[16], with each thread exclusively updating its own element partial[tid]. Performance is unexpectedly poor. Which architectural phenomenon is the primary cause?

(A) Thread migration between cores causes cache flushes.
(B) The compiler placed the array on a single memory page, causing page faults.
(C) The whole line is invalidated in other caches called false sharing.
(D) The memory controller serialises accesses to the same DRAM bank.
(E) OpenMP's implicit barrier after every loop iteration.

> **Solution:** C
> Reasoning: False sharing occurs when two threads write to different memory locations that happen to reside on the same cache line. The coherence protocol treats any write to the line as invalidating the entire line, causing excessive cache misses and coherence traffic. Padding to align each element to a separate cache line solves this.

(18) **(1 Mark)** A Halide schedule includes .vectorize(x, 8) for an image processing pipeline that operates on float pixels. The compiler generates AVX2 instructions (256-bit vectors, 8 floats). If the same program is compiled for a machine with AVX-512 (512-bit vectors, 16 floats) without changing the schedule, what will happen?

(A) The program will not run because the vector width is hardcoded.
(B) The compiler will automatically use the wider vectors because the schedule only specifies the vectorisation dimension, not the width; the actual width is determined by the target architecture.
(C) The program will run but only use 8-way vectorisation, wasting half the vector registers.
(D) The schedule must be changed to .vectorize(x, 16).
(E) AVX-512 is not supported by Halide.

> **Solution:** C

(19) **(1 Mark)** A checkpoint/restart system saves the memory image of a parallel application. On a UMA machine, checkpointing a large array is fast because the array is contiguous in physical memory. On a cc-NUMA machine, the same array is distributed across many nodes' local memories. What architectural implication does this have for checkpoint performance?

(A) Checkpointing on NUMA is faster because each node can save its part in parallel.
(B) Checkpointing on NUMA is slower because the checkpointing library must gather all remote memory through the interconnect, potentially saturating it.
(C) There is no difference because all memory is equally accessible.
(D) Checkpointing is impossible on NUMA because memory is not globally addressable.
(E) NUMA machines have built-in hardware checkpointing, so no software overhead.

> **Solution:** B
> Reasoning: In UMA, the checkpoint library can read the whole array from a single memory controller. In NUMA, the array is physically striped; a single-threaded checkpoint process would incur many remote accesses, saturating the interconnect. A well-designed NUMA checkpoint uses parallel, per-node I/O (each node saves its local portion) to avoid this.

(20) **(1 Mark)** A parallel loop has 10,000 iterations, each taking a highly variable amount of time (some iterations are 1000× slower than others). The programmer tries three schedules on 32 threads:

- schedule(static): poor performance due to load imbalance

- schedule(dynamic,1): good load balance but high scheduling overhead
- schedule(guided,10): best performance overall

What architectural reason explains why guided with chunk size 10 outperforms dynamic,1?

(A) Guided starts with larger chunks to reduce overhead, then dynamically switches to smaller chunks to balance the tail, adapting to load variation without excessive per-iteration scheduling cost.
(B) Guided uses work stealing in hardware, while dynamic relies on software scheduling.
(C) Dynamic scheduling with chunk size 1 causes each iteration to be assigned individually, increasing the number of scheduler calls to 10,000, which saturates the memory bus.
(D) Guided uses a global lock only once per thread, while dynamic acquires a lock per chunk, causing serialisation.
(E) Guided is implemented in hardware on modern CPUs, while dynamic is a software fallback.

Solution: A
Reasoning: Guided scheduling starts with large chunks (proportional to remaining iterations / threads) and exponentially decreases chunk size. This reduces overhead (fewer chunks) while still balancing the tail. Dynamic with chunk size 1 has maximum overhead (one scheduler call per iteration). Option (C) is partially true but overstates – the memory bus is not saturated; the overhead is in the scheduler's critical section. Option (B) – work stealing is not typically in hardware. Option (D) – both use locks, but guided has far fewer chunks. Option (E) – both are software.

(21) **(1 Mark)** In a distributed-memory HPC cluster, a programmer implements a ring-based halo exchange using the standard Message Passing Interface (MPI). Process N attempts to send boundary data to Process $N+1$ using a standard blocking send (MPI_Send), and immediately afterward attempts to receive data from Process $N-1$ using a standard blocking receive (MPI_Recv). All processes in the communicator execute this exact same code structure simultaneously. Which statement accurately describes the runtime behavior of this implementation as the message payload size scales?
A) The code will execute flawlessly at all scales because MPI_Send is fundamentally asynchronous by specification and guarantees immediate return to the calling pipeline.
B) The execution will complete successfully for small message payloads due to internal MPI system buffering, but will reliably deadlock for large payloads when the message size exceeds the eager-protocol buffer limit.
C) A permanent deadlock will occur instantly for all message sizes because standard blocking communication primitives strictly forbid concurrent execution across distinct cluster nodes.
D) The MPI runtime environment will automatically detect the circular dependency and transparently upgrade the operations to MPI_Isend and MPI_Irecv to prevent a pipeline stall.
E) The application will suffer an immediate segmentation fault because executing MPI_Send before MPI_Recv inside an identical code block violates the single-program multiple-data (SPMD) sequencing standard.

Solution: B
Explanation: Standard MPI_Send can behave as either buffered or synchronous depending on the message size. For small messages, MPI uses the Eager Protocol, copying the data to a local system buffer and returning immediately, allowing the code to reach the MPI_Recv and finish safely. For large messages, MPI switches to the Rendezvous Protocol, where MPI_Send blocks until a matching receive is posted. Since every process is blocked trying to send before anyone can call receive, the entire ring permanently deadlocks.

(22) **(1 Mark)** Consider the following nested loop executing sequentially in a C program. An optimizing compiler analyzes this loop to evaluate whether it can safely apply loop transformations such as loop interchange (swapping the i and j loops to improve spatial locality) or full loop vectorization:

```
1    for (int i = 1; i < N; i++) {
2        for (int j = 1; j < N; j++) {
3            A[i][j] = A[i-1][j+1] + 5.0;
4        }
5    }
```

By computing the distance vector $D = (d_i, d_j)$ and direction vector $\Delta = (\delta_i, \delta_j)$ for the data dependencies present in this loop, which statement accurately characterizes the validity of these compiler optimizations?
A) The distance vector is $(1, -1)$ and the direction vector is $(<, >)$, indicating a loop-carried dependence that permits safe loop interchange but strictly prevents inner-loop vectorization.
B) The distance vector is $(-1, 1)$ and the direction vector is $(>, <)$, which represents an illegal lexicographically negative dependence, forcing the compiler to abort compilation due to undefined behavior.
C) The distance vector is $(1, 1)$ and the direction vector is $(<, <)$, meaning a true flow dependence exists across both dimensions, allowing the compiler to safely perform both loop interchange and full vectorization natively.
D) The direction vector is $(<, >)$, which means a loop-carried dependence exists. Swapping the loops modifies the direction vector to $(>, <)$, making it lexicographically negative; therefore, loop interchange is illegal, but the inner loop can be vectorized.
E) The loop contains a anti-dependence with a distance vector of $(1, 0)$, which creates a loop-independent hazard that completely blocks both loop interchange and loop unrolling unless explicit array renaming is applied.

Solution: D
If you swap the loops so j is the outer loop, the direction vector flips to $(>, <)$. A direction vector whose leftmost non-zero entry is "$>$" means a loop depends on a future iteration that hasn't happened yet. This violates sequential correctness and makes loop interchange illegal.

(23) **(1 Mark)** In a parallel sorting algorithm implemented with 8 processes, each process computes a local minimum of its data partition. The global minimum value needs to be known by ALL processes to determine the next pivot element for the next iteration. The algorithm iterates 1000 times. Which communication pattern provides the best performance?
A) Use MPI_Reduce to compute the global minimum at root (process 0), then use MPI_Bcast to broadcast the result to all processes in each iteration
B) Use point-to-point communication to send all local minima to process 0, have process 0 compute the minimum, then send back to all processes
C) Use MPI_Scatter to distribute the minima to all processes and MPI_Gather to collect the results
D) Use MPI_Allreduce directly in each iteration to compute the global minimum and deliver it to all processes
E) None of the above

Solution: D

(24) **(1 Mark)** Consider the following MPI non-blocking communication code fragment executed by two distinct processes:

```
// Assume standard initialization and MPI_COMM_WORLD environment
// --- Process 0 ---
if (rank == 0) {
    int data = 100;
    MPI_Request req;
    data = 200; // Buffer is modified first
    MPI_Isend(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &req);
    MPI_Wait(&req, MPI_STATUS_IGNORE);
}
// --- Process 1 ---
if (rank == 1) {
    int received = 0;
    MPI_Recv(&received, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("%d", received);
}
```

What value is to be printed by Process 1?
A) 100

B) 200
C) Undefined (race condition)
D) The program will crash
E) None of the above

Solution: B

(25) **(1 Mark)** A naive OpenCL implementation of matrix multiplication where each work-item computes a single output element performs poorly on a GPU compared to a sequential CPU version. What is the PRIMARY architectural reason for this performance degradation?
A) The GPU has fewer compute units than the CPU core count
B) GPU should not be used for single element computation
C) The OpenCL compiler cannot optimize loops when using 2D global IDs
D) Barrier synchronization between work-groups causes deadlock
E) Each work-item independently fetches overlapping data from global memory, causing excessive bandwidth utilization

Solution: E

(26) **(1 Mark)** Consider the following OpenCL kernel declaration:

```
1    __kernel void process(__global float* input, __local float* cache,
2    __private float scale)
```

Which statement correctly describes the storage location and accessibility of the three parameters?
A) input is in host memory, cache is in global memory, scale is in constant memory
B) input is read-only, cache is shared across all work-groups, scale is shared within a work-group
C) input is accessible by all work-groups, cache is shared within a work-group, scale is private to each work-item
D) input is in local memory, cache is in private memory, scale is in global memory
E) None of the abvove seems correct

Solution: C

(27) **(1 Mark)** In a tiled OpenCL kernel that uses __local memory to cache data for a work-group, what is the consequence of removing all barrier(CLK_LOCAL_MEM_FENCE) calls?
A) The kernel will produce correct results but with degraded performance because work-items will redundantly fetch data from global memory instead of reusing local cache
B) The kernel may produce incorrect results because some work-items could read stale or partially written local memory values, though results might occasionally appear correct depending on scheduling order
C) The kernel will fail to compile because the OpenCL specification requires at least one barrier in any kernel that declares a __local variable
D) The kernel will produce correct results for some input sizes but produce incorrect results for others, depending on whether the work-group size divides evenly into the global size
E) None of the above

Solution: B

(28) **(1 Mark)** Consider the following loop:

```
DO I = 120, 30, -15
A[I] = B[I + 15] + C[I / 3]
ENDDO
```

if this loop is normalized to an iteration variable i that starts at 1 and increments by 1, what is the correct upper bound and the transformed array access expression for A?
A) Upper bound = 6; Array access = A[135 - 15*i]
B) Upper bound = 7; Array access = A[120 - 15*i]

C) Upper bound = 7; Array access = A[135 - 15*i]
D) Upper bound = 6; Array access = A[120 - 15*i]
E) None of the above

Solution: C

(29) **(1 Mark)** Determine whether loop interchange is valid for the following loop nest:
```
DO I = 1, N
DO J = 1, M
A(I+1, J-1) = A(I, J) + B(I, J)
ENDDO
ENDDO
```
A) Direction vectors: (<, >); interchange is invalid because it would change the direction vector to (>, <)
B) Direction vectors: (<, >); interchange is valid because the direction vector remains lexicographically positive after interchange
C) Direction vectors: (=, <); interchange is valid because only the inner loop carries the dependence
D) Direction vectors: (<, =); interchange is valid because only the outer loop carries the dependence
E)None of the above

Solution: A

(30) **(1 Mark)** In a shared-memory OpenMP program, checkpoint/restart can be implemented by saving the entire process's memory image. In a distributed-memory MPI program, coordinated checkpoint/restart requires each rank to save its own memory image, and the restart must load all images consistently. Which of the following correctly describes an additional challenge that MPI checkpointing faces but OpenMP checkpointing does not?

(A) MPI checkpoints must use non-blocking I/O to avoid deadlock, while OpenMP can use blocking I/O.
(B) In MPI, the checkpoint operation must include a global barrier or consensus to ensure all ranks save state from the same logical iteration; otherwise, a restart could load an inconsistent global state.
(C) OpenMP checkpoints require atomic operations to avoid data races, while MPI checkpoints do not because each rank has private memory.
(D) MPI checkpoints must be stored on a shared file system, while OpenMP checkpoints can use local disk, making MPI inherently slower.
(E) OpenMP cannot checkpoint at all because multiple threads share a single address space, so any checkpoint would include the entire application state.

Solution: B
Reasoning: In distributed memory, ranks operate on different portions of data. Without coordination (e.g., a barrier before each checkpoint), one rank might save state at iteration 100 while another saves at iteration 101. Upon restart, the global state would be inconsistent. OpenMP shares a single address space; a single process checkpoint inherently captures a consistent snapshot (assuming no external communication).

Good Luck!

MCQs Answer Sheet

Instruction for filling the sheet
1. This sheet should not be folded or crushed
2. Use only blue/black ball pen or marker.
3. Circle should darkened completely and properly.
4. Erase marked circle completely for deselect
5. MCQs will only be marked using this OMR sheet.

NAME :

Flex ID: :

Section: :